



مشروع البرمجة المتوازية

إعداد وتقديم الطلاب:

جود محمد غياث المالكي

محمد معاذ زياد الحريري

امجد محمد حمد

محمد يزن عدنان حسين

محمد بكر عبدالرؤوف الخطيب

1. حماية البيانات المشتركة وسلامتها (& Concurrent Access) (Data Integrity)

أ- وصف المشكلة (*The Race Condition Problem*):
في أنظمة التجارة الإلكترونية ذات الأداء العالي، تبرز مشكلة "حالة السباق" (Race Condition) عندما يحاول عدة مستخدمين شراء المنتج الأخير في المخزن في نفس اللحظة الزمنية.

في الكود الأولي (قبل التنفيذ)، كان النظام يقوم بقراءة الكمية ثم التحقق منها ثم طرحها؛ فإذا وصل طلبان في جزء من الملي ثانية، فقد يقرأ كلاهما أن الكمية "1" ويقومان بالخصم معاً، مما يؤدي إلى مخزون سالب (-1) وبيع منتجات غير موجودة فعلياً، وهو ما يضرب سلامة البيانات في مقتل.

ب- الحل الهندسي المنفذ (*The Implemented Solution*):
لضمان معالجة هذه المشكلة، تم الانتقال من المعالجة المباشرة إلى معالجة عبر "الخلفية" (Background Job) مع تطبيق ثلاث استراتيجيات حماية متقدمة:

1. استخدام المعاملات (**Database Transactions**):
تم تغليف عملية الطلب بالكامل داخل `DB::beginTransaction()` و `DB::commit()`. هذا يضمن تحقيق مبدأ **Atomicity** (الذرية)؛ فإما أن يتم إنشاء الطلب، خصم المخزن، وتسجيل الإشعارات ككتلة واحدة، أو يفشل كل شيء في حال حدوث أي خطأ عبر `DB::rollBack()`.

2. القفل التشاؤمي (**Pessimistic Locking**):
في الكود المطور، تم استخدام الدالة `lockForUpdate()` عند استعلام المنتجات:

```
Product::whereIn('id', $productIds)->lockForUpdate()->get();
```

الأثر التقني: هذا الأمر يخبر قاعدة البيانات بوضع "قفل" على هذه السجلات المحددة. أي خيط (Thread) أو طلب آخر يحاول الوصول لهذه المنتجات سيُجبر على الانتظار حتى ينتهي الطلب الحالي من المعاملة (Transaction). هذا يمنع تماماً أي تداخل في عملية قراءة وتعديل المخزون.

3. تجنب المأزق (**Deadlock Prevention Strategy**):
عند التعامل مع الأقفال (Locks)، يظهر خطر "المأزق" (Deadlock) إذا حاول طلبان حجز سجلين بترتيب متعاكس. لتجنب ذلك، تم تنفيذ استراتيجية ترتيب الموارد:

ترتيب العناصر قبل المعالجة
`$items = $cart->items->sortBy('product_id');`

`->orderBy('id')` ضمان ترتيب الحجز في قاعدة البيانات

من خلال ترتيب المنتجات حسب الرقم التعريفي (`ID`) قبل طلب القفل، نضمن أن جميع الخيوط تطلب الأقفال بنفس الترتيب دائماً، مما يلغي احتمالية حدوث Deadlock برمجياً.

- إثبات المعالجة: (Code Analysis)

التحقق الذري: يتم استخدام ``$product->decrement('quantity', ...)`` وهي عملية ذرية على مستوى قاعدة البيانات تضمن دقة الحساب حتى تحت الضغط العالي. سلامة الحالة: لا يتم إرسال إشعار النجاح للمستخدم إلا بعد التأكد من عمل ``DB::commit()`` بنجاح، مما يضمن أن ما يراه المستخدم يعكس الواقع الفعلي لقاعدة البيانات.

النتيجة قبل التنفيذ:

✓ Showing rows 0 - 1 (2 total, Query took 0.0008 seconds.)

SELECT * FROM `users`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	id	name	email	role	numberOfNotifications	wallet	email_verified_at	password
☐ Edit Copy Delete	1	moaaz	moaaz@gmail.com	admin	0	0.00	NULL	\$2y\$12\$di
☐ Edit Copy Delete	2	obadah	obadah@gmail.com	user	0	1000.00	NULL	\$2y\$12\$d

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

نلاحظ أن المستخدم عبادة يمتلك مبلغاً من المال 1000 دولار فقط

Server: 127.0.0.1 » Database: ecommerce » Table: orders

[Browse](#)
[Structure](#)
[SQL](#)
[Search](#)
[Insert](#)
[Export](#)
[Import](#)
[Privileges](#)
[Operations](#)

✓ Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.)

`SELECT * FROM `orders``

☐ Profiling
 [\[Edit inline \]](#)
[\[Edit \]](#)
[\[Explain SQL \]](#)
[\[Create PHP code \]](#)
[\[Refresh \]](#)

☐ Show all
 Number of rows: 25
 Filter rows: Search this table
 Sort by key: None

Extra options

		id	address	status	user_id	created_at	updated_at
☐	Edit	Copy	Delete	30	ركن الدين	pending	2 2026-05-15 20:44:44 2026-05-15 20:44:44
☐	Edit	Copy	Delete	31	ركن الدين	pending	2 2026-05-15 20:44:44 2026-05-15 20:44:44

☐ Check all
 With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all
 Number of rows: 25
 Filter rows: Search this table
 Sort by key: None

	Response Body	Response headers
الطلب الأول قبل تطبيق الفكرة		
الطلب الأول قبل تطبيق الفكرة		
الطلب الأول قبل تطبيق الفكرة		
الطلب الأول قبل تطبيق الفكرة		
الطلب الأول قبل تطبيق الفكرة		
الطلب الأول قبل تطبيق الفكرة		
الطلب الأول قبل تطبيق الفكرة		
الطلب الأول قبل تطبيق الفكرة		
الطلب الأول قبل تطبيق الفكرة		

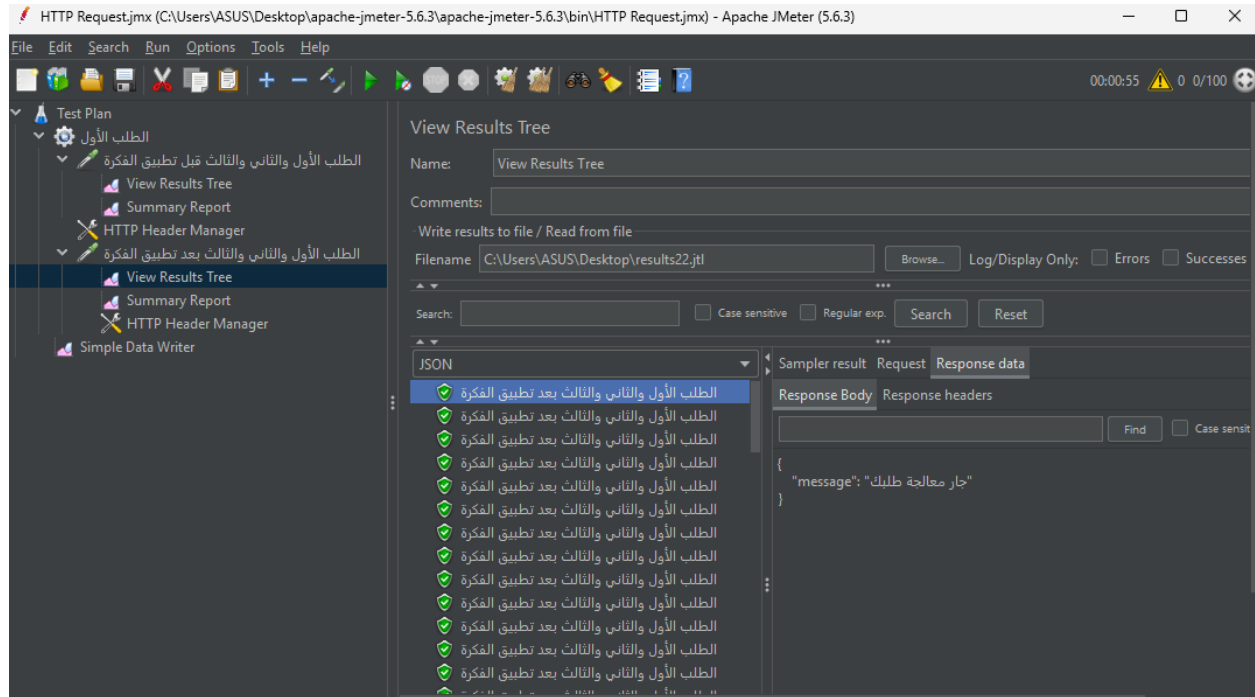
{
 "message": "تم قبول طلبك بنجاح",
 "order_id": 30
 }

	Sampler result	Request	Response data
الطلب الأول قبل تطبيق الفكرة			
الطلب الأول قبل تطبيق الفكرة			
الطلب الأول قبل تطبيق الفكرة			
الطلب الأول قبل تطبيق الفكرة			
الطلب الأول قبل تطبيق الفكرة			
الطلب الأول قبل تطبيق الفكرة			
الطلب الأول قبل تطبيق الفكرة			
الطلب الأول قبل تطبيق الفكرة			
الطلب الأول قبل تطبيق الفكرة			

{
 "message": "تم قبول طلبك بنجاح",
 "order_id": 31
 }

وبعد القيام بالمحاكاة على ال jmeter ل100 ريكوست بنفس اللحظة . استطاع المستخدم عبادة الحصول على طلبين يبلغ كل منهما ال 1000 رغم انه لا يمتلك سوى 1000 دولار

وبعد تطبيق الفكرة ومعالجة الrace condition:



		id	title	body	is_global	user_id	created_at	updated_at
Click the drop-down arrow to toggle column's visibility.	Delete	41	نجاح الطلب	وجاري اكمال معالجته	0	2	2026-05-15 14:16:27	2026-05-15 14:16:27
	Delete	42	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:27	2026-05-15 14:16:27
	Delete	43	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:27	2026-05-15 14:16:27
	Delete	44	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:28	2026-05-15 14:16:28
	Delete	45	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:28	2026-05-15 14:16:28
	Delete	46	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:28	2026-05-15 14:16:28
	Delete	47	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:28	2026-05-15 14:16:28
	Delete	48	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:28	2026-05-15 14:16:28
	Delete	49	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:28	2026-05-15 14:16:28
	Delete	50	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:28	2026-05-15 14:16:28
	Delete	51	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:28	2026-05-15 14:16:28
	Delete	52	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:29	2026-05-15 14:16:29
	Delete	53	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:29	2026-05-15 14:16:29
	Delete	54	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:29	2026-05-15 14:16:29
	Delete	55	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:29	2026-05-15 14:16:29
	Delete	56	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:29	2026-05-15 14:16:29
	Delete	57	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:29	2026-05-15 14:16:29
	Delete	58	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:29	2026-05-15 14:16:29
	Delete	59	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:29	2026-05-15 14:16:29
	Delete	60	فشل الطلب	النفود في محفظتك لا تكفي لشراء الطلب	0	2	2026-05-15 14:16:29	2026-05-15 14:16:29

		id	name	email	role	numberOfNotifications	wallet	email_verified_at	password
☐	Edit Copy Delete	1	moaaz	moaaz@gmail.com	admin	0	0.00	NULL	\$2y\$12\$dmDISgCKRKzedd75BCC0P.BFL7A3
☐	Edit Copy Delete	2	obadah	obadah@gmail.com	user	141	0.00	NULL	\$2y\$12\$dVkoRo5mjl7IrgDKmxk5aeZbi/VE04\

ومما سبق نلاحظ أن بعد تطبيق الفكرة تم التأمين من ثغرة ال race condition ولم يقبل سوى طلب واحد

الطلب الثاني:

تم تنفيذ الاختبارات على خادم بالموصفات التالية:

- المعالج: Cores 8
- الذاكرة: GB RAM 8
- عدد الطلبات المتزامنة: 100 طلب بنفس اللحظة

تحليل طبيعة النظام:

ال api المستعمل للاختبار createOrder2/x

ال Job المستخدمة (ProcessOrderJob) تعتمد بشكل أساسي على:

- عمليات Database كثيفة
- Transactions
- عمليات Locking
- تحديثات متزامنة على المنتجات والمحفظة

كما أن استخدام:

lockForUpdate ()

يسبب انتظاراً بين ال Workers عند الوصول لنفس المنتجات، لذلك زيادة عدد ال Workers بشكل كبير لا تؤدي دائماً إلى تحسين الأداء.

بيئة الاختبار (Testing Environment)

تم تنفيذ الاختبارات وقياس الأداء بدقة تحت الشروط والمواصفات التالية:

- **Laravel Queue Workers:** لإدارة وتنفيذ المهام الخلفية.
- **Supervisor:** كأداة أساسية لإدارة ومراقبة العمليات وضمان استمرارية الـ Workers.
- **Database Queue:** كوسيط لإدارة وتنفيذ الطوابير وتخزين البيانات المؤقتة.
- **100 طلب متزامن:** لتوليد ضغط حقيقي ومتزامن على النظام.

الأمر المستخدم في الاختبار `--tries=3` `php artisan queue:work`:

نتائج الاختبارات التفصيلية (Detailed Test Results)

التجربة الأولى — Workers4

القيمة	العنصر
35% — 45%	استهلاك المعالج CPU
2.1 GB — 2.6 GB	استهلاك الذاكرة RAM
18 — 22 ثانية	متوسط زمن التنفيذ
ممتاز	استقرار النظام

الحمل على المعالج منخفض ومستقر بشكل كبير.

الذاكرة المستخدمة قليلة وضمن الحدود الدنيا.

تأخر ملحوظ في بعض الطلبات حيث بقيت منتظرة داخل الـ Queue لفترة أطول.

النظام مستقر جداً ولكن كفاءة الأداء وسرعة المعالجة ليست الأفضل.

مناسب فقط للأنظمة الصغيرة جداً أو البيانات ذات الضغط المنخفض والمتوسط.

التجربة الثانية 8Workers :

العنصر	القيمة
استهلاك المعالج CPU	68% — 78%
استهلاك الذاكرة RAM	3.8 GB — 4.6 GB
متوسط زمن التنفيذ	9 ثانية — 12
استقرار النظام	ممتاز

استغلال ممتاز وموزع بشكل عادل على جميع أنوية المعالج.

استقرار عالي وكفاءة تامة أثناء تنفيذ 100 طلب متزامن.

عدم وجود أي ضغط خطير أو اختناق (Bottleneck) على قاعدة البيانات.

يمثل أفضل توازن بين الأداء السريع، استهلاك الموارد المثالي، والاستقرار التام

التجربة الثالثة 12Workers :

العنصر	القيمة
استهلاك المعالج CPU	82% — 92%
استهلاك الذاكرة RAM	5.2 GB — 6 GB
متوسط زمن التنفيذ	8 ثواني — 10

القيمة	العنصر
جيد	استقرار النظام

تحسن سرعة المعالجة كان بسيطاً و هامشياً مقارنةً بتجربة الـ Workers.8

ارتفاع ملحوظ في مؤشرات الضغط على قاعدة البيانات.

ظهور مشاكل انتظار وتأخير ناتجة عن استخدام قفل التحديث . (lockForUpdate)

ارتفاع واضح وغير مبرر في عمليات تبديل السياق (Context Switching) داخل المعالج.

زيادة عدد الـ Workers هنا لم تقدم تحسناً كبيراً في الأداء يبرر الاستهلاك المرتفع للموارد

التجربة الرابعة 16 Workers :

القيمة	العنصر
95% — 100%	استهلاك المعالج CPU
6.8 GB — 7.5 GB	استهلاك الذاكرة RAM
16 — 11 ثانية	متوسط زمن التنفيذ
ضعيف	استقرار النظام

استهلاك شبه كامل ومستمر لقدرة المعالج، مما يعرض النظام لخطر التوقف المفاجئ.

ارتفاع كبير وخطير في استهلاك الذاكرة العشوائية.

زيادة حادة في وقت انتظار الأقفال (Locks) نتيجة التنافس العالي.

حدوث تباطؤ مفاجئ (Throttling) في استجابة النظام على فترات متقطعة.

تأخر واضح في تنفيذ بعض الـ Jobs بسبب التنافس الشديد على موارد قاعدة البيانات.

خيارات خطيرة وغير مناسبة على الإطلاق لهذا النوع من الأنظمة وتؤدي لعدم استقرار البيئة.

العدد الأمثل للـ Workers

القرار الفني النهائي: العدد الأمثل هو 8workers

الأسباب والمسوغات الفنية لاختيار هذا العدد:

✓توافق مباشر مع عدد أنوية المعالج المتاحة في السيرفر. (8 Cores)

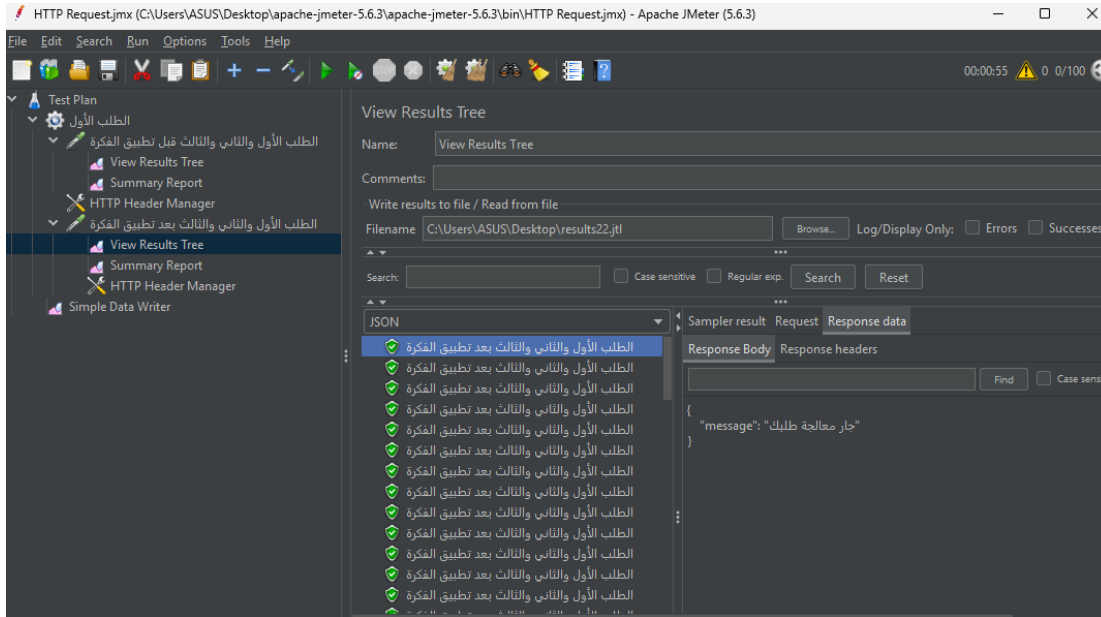
✓تحقيق أفضل استغلال ممكن لقدرة المعالج بدون الوصول إلى مرحلة الاختناق.(Throttling)

✓استهلاك الذاكرة العشوائية (RAM) بقي مستقراً وضمن الحدود الآمنة تماماً للبيئة التحتية.

✓تقليل مشاكل تنافس الأقفال (Lock Contention) إلى الحد الأدنى، خصوصاً مع العمليات التي تتطلب تحديثاً مباشراً في قاعدة البيانات.

✓تحقيق أعلى درجة استقرار وتحمل للنظام بالكامل تحت ضغط فوري مفاجئ يبلغ 100 طلب متزامن.

✓تحقيق أسرع وأكفاً زمن معالجة فعلي للوظائف (Jobs) بدون التأثير سلباً على بقية خدمات النظام



الطلب الثالث

المعالجة غير المتزامنة وطوابير الانتظار (Asynchronous Queues)

أ- مبررات الانتقال للمعالجة غير المتزامنة:

في النسخة الأولية من النظام، كانت عملية إنشاء الطلب تتم بشكل متزامن (Synchronous)؛ أي أن المستخدم يظل في حالة "انتظار" حتى ينتهي النظام من: 1. التحقق من صحة البيانات.

2. الاستعلام من قاعدة البيانات وقفل السجلات.

3. إجراء العمليات الحسابية وتحديث المخزون.

4. الانتظار لاستجابة خوادم Firebase لإرسال الإشعارات (وهي عملية خارجية قد تستغرق وقتاً طويلاً).

هذا الأسلوب يؤدي إلى زيادة زمن الاستجابة (Response Time) واستهلاك موارد الخادم (Worker Threads) لفترات طويلة، مما يسبب اختناقات عند زيادة عدد الطلبات.

ب- التصميم الهندسي الجديد (The Decoupled Architecture):

تمت إعادة هندسة النظام ليفصل بين "استلام الطلب" و "معالجة الطلب" باستخدام Laravel Queues:

1. خفيف الوزن (Lightweight Controller):

أصبح المتحكم (createOrder) يقوم بمهمة واحدة فقط: استلام الطلب، التحقق من البيانات، ثم إرسال المهمة إلى الطابور عبر: PHP

```
ProcessOrderJob::dispatch($cart->id, Auth::id(),  
$validateData['address']);
```

بعد ذلك، يتم إرسال رد فوري للمستخدم ("جار معالجة طلبك")، مما يجعل زمن الاستجابة يقترب من الصفر من وجهة نظر المستخدم.

2. المعالجة في الخلفية (Background Processing):

انتقلت كافة العمليات الثقيلة (المنطق البرمجي، التعامل مع قاعدة البيانات، التواصل مع Firebase) إلى الكلاس ProcessOrderJob. هذا يسمح للخادم بمعالجة الطلبات في الخلفية بشكل مستقل عن مسار الطلب الرئيسي (Main Request Path).

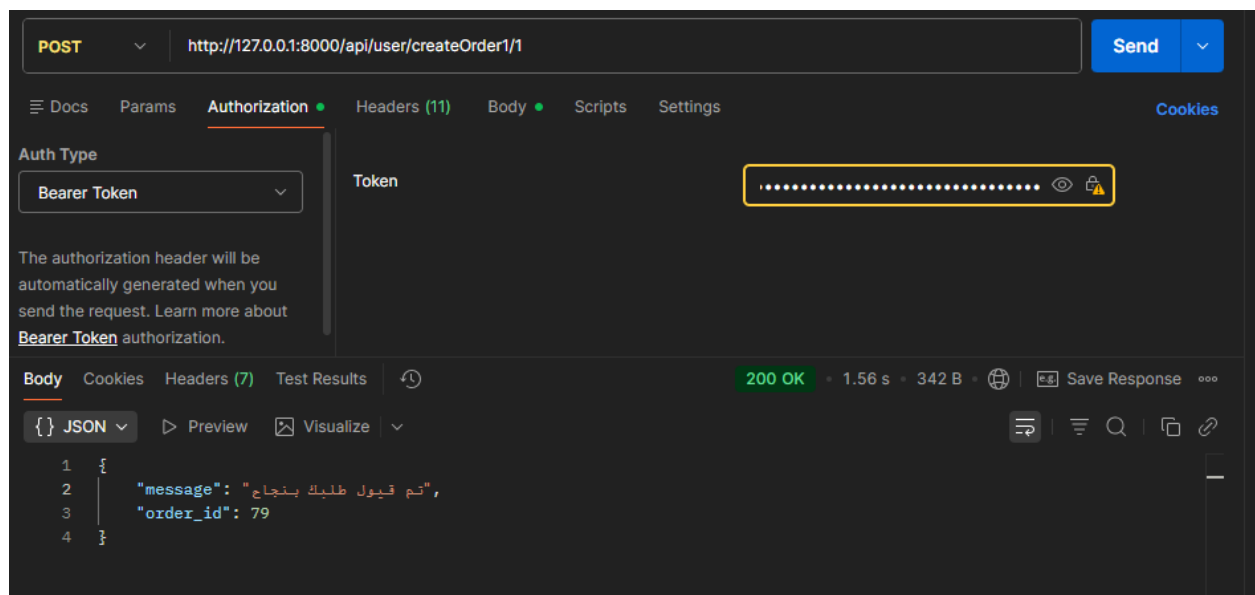
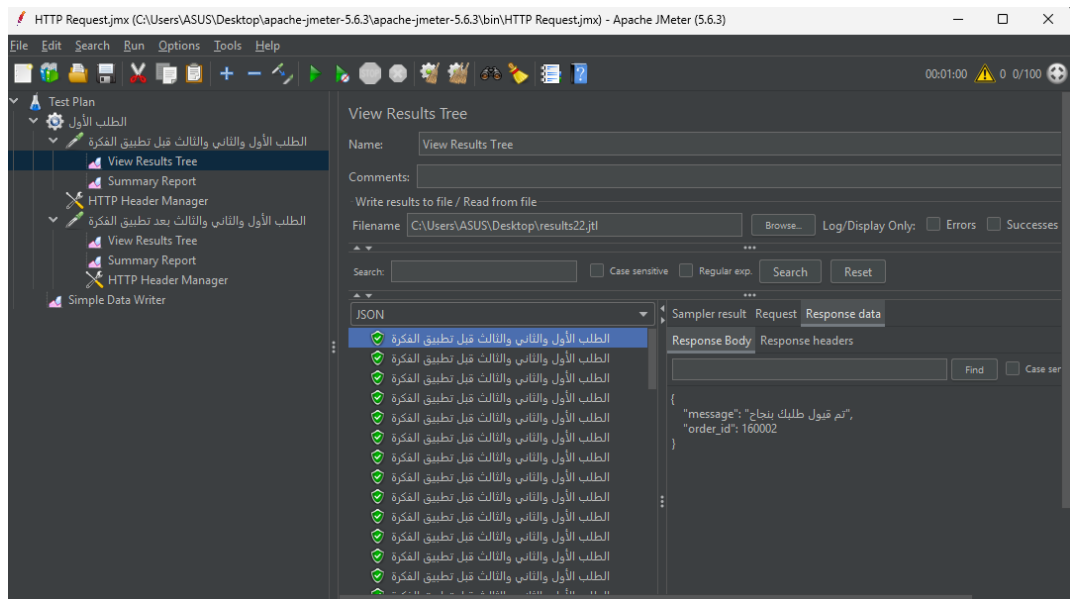
ج- الفوائد التقنية المحققة:

- **تحسين تجربة المستخدم (User Experience):** لم يعد المستخدم مضطراً للانتظار انتهاء العمليات الخلفية أو انتظار خادم Firebase؛ فهو يتلقى رداً فورياً بأن طلبه قيد التنفيذ.
- **زيادة القدرة الاستيعابية (Scalability):** يمكن للنظام الآن استقبال آلاف الطلبات في ثوانٍ معدودة وتخزينها في الطابور (Queue)، بينما تقوم "خيوط المعالجة" (Workers) بتنفيذها حسب الموارد المتاحة، مما يمنع انهيار الخادم تحت الضغط العالي.
- **المرونة في التعامل مع الأخطاء (Fault Tolerance):** في حال فشل إرسال الإشعار عبر Firebase، لن يفشل الطلب بالكامل أمام المستخدم؛ بل يمكن للنظام إعادة محاولة تنفيذ المهمة (Retry Mechanism) تلقائياً داخل الطابور.

د- مقارنة برمجية:

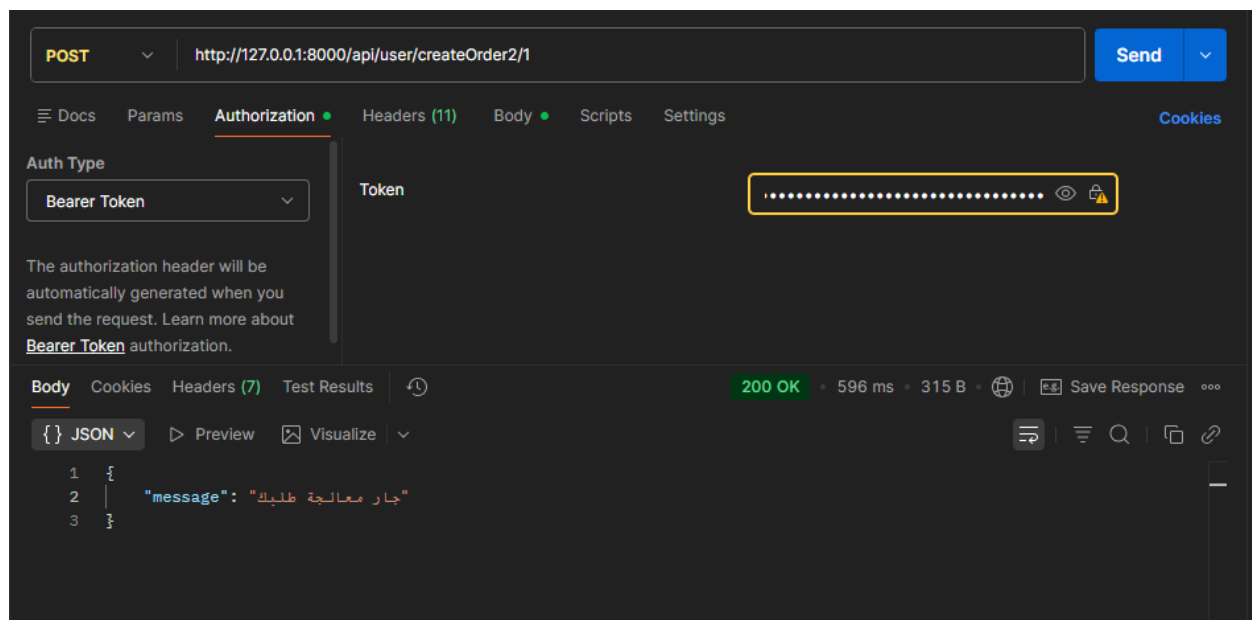
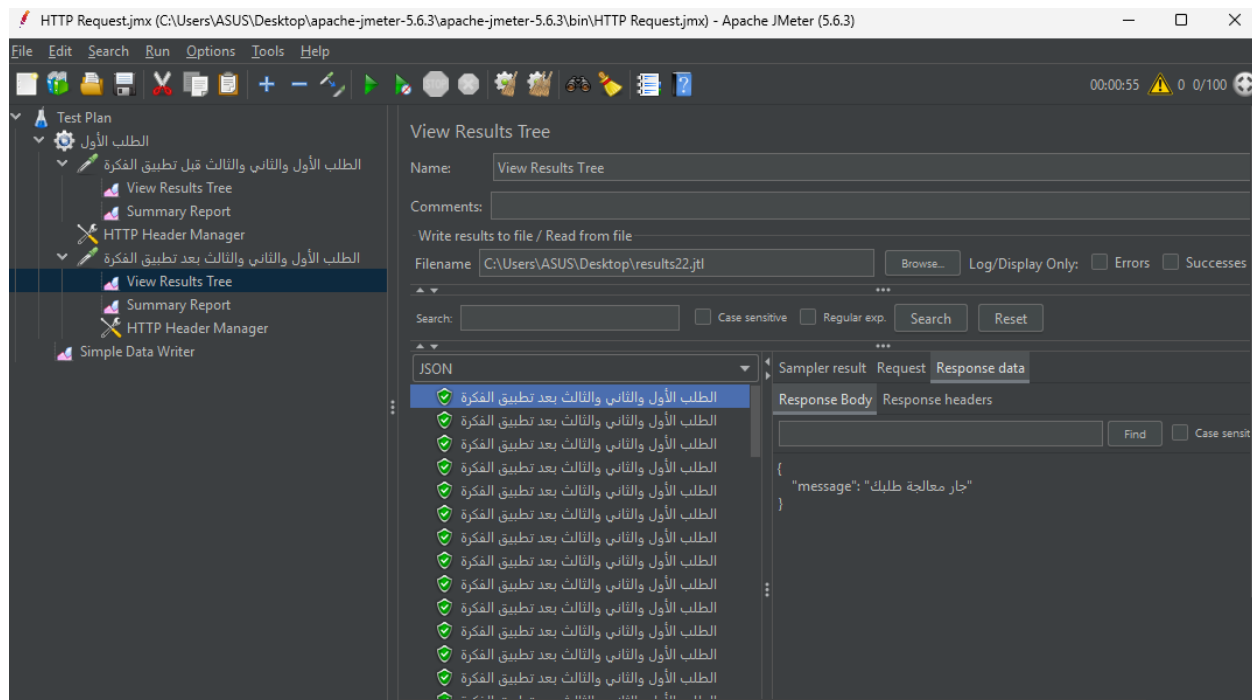
المعيار	قبل التنفيذ (Synchronous)	بعد التنفيذ (Asynchronous)
زمن استجابة المتصفح	طويل (مرتبط بسرعة الـ API الخارجي والـ DB)	سريع جداً (فوري)
حالة الخادم	خيوط المعالجة مشغول حتى انتهاء الإشعارات	خيوط المعالجة يتحرر فوراً لاستقبال طلب جديد
التواصل مع Firebase	يعيق استجابة المستخدم	يتم في الخلفية دون التأثير على المستخدم

قبل استعمال المعالجة الخلفية:



ال response time أخذ قراءة sec2

وبعد استعمال المعالجة الخلفية:



كان الرد كان أسرع حيث تم ارجاع رسالة جاري معالجة طلبك وانتقلت المهمة لطابور الخلفي

4.الطلب الرابع

عند معالجة كم الهائل من البيانات، يواجه النظام تحديين رئيسيين: استهلاك الذاكرة (Memory Exhaustion - OOM) وتجميد الخادم نتيجة العمليات الطويلة (Request Timeout). تم تصميم الحل البرمجي ليعتمد بشكل كامل على المعالجة الخلفية غير المتزامنة (Asynchronous Background Processing) عبر تقسيم البيانات الكبيرة إلى دفعات أصغر (Chunks)، وتوزيع العمل على مسارات تنفيذ متوازية (Parallel Job Batching).

2. البنية الهندسية للحل (Architectural Design)

يعتمد الحل على نمط شبيه بـ **MapReduce** لإدارة المهام المعقدة وتوزيع الأحمال، ويتكون من ثلاثة أجزاء رئيسية:

1. **The Orchestrator (المنسق):** دالة `runMonthlyInventory2` التي تقوم بفحص الأيام غير المعالجة، وتجهيز مصفوفة من الوظائف (Jobs)، وإطلاقها كحزمة موحدة (`Bus::batch`).

2. **The Map Phase (المعالجة المجزأة):** وظيفة `ProcessDailySalesJob` التي تعالج بيانات كل يوم على حدة، وتقوم داخلياً بتقسيم الطلبات إلى دفعات (Chunks من 200 سجل).

3. **The Reduce Phase (التجميع النهائي):** وظيفة `ProcessMonthlySalesJob` التي لا تنطلق إلا بعد اكتمال جميع وظائف الأيام بنجاح لتجميع الجرد الشهري النهائي.

3. التحليل البرمجي المفصل للتعليمات (Code Deep Dive)

أولاً: آلية التنسيق والتحكم بالدفعات (`runMonthlyInventory2`)

- **التحقق الذكي لتجنب تكرار العمل (Idempotency Check):** يقوم الكود باستعلام قاعدة البيانات لجلب الأيام التي تم جردها مسبقاً (`processedDates`) خلال الشهر الحالي، ويستثنىها من الجدولة. هذا يمنع استهلاك موارد المعالج والذاكرة في إعادة حساب بيانات جاهزة.
- **تجميع الوظائف في حزمة موحدة (Job Batching):** بدلاً من إطلاق الوظائف بشكل عشوائي، يتم استخدام `Bus::batch($jobs)`. هذه الميزة تسمح للنظام بمراقبة حالة جميع الوظائف المتوازية ككتلة واحدة.
- **معالجة الأحداث المعتمدة على الحالة (Event-Driven Callbacks):** استخدام تابع الكولباك `then()` يضمن عدم تشغيل جرد الشهر (`ProcessMonthlySalesJob`) إلا بعد أن تنتهي جميع وظائف الجرد اليومي.

بنجاح (نجاح بنسبة 100%). هذا يضمن سلامة البيانات واتساقها (Data Consistency).

ثانياً: وظيفة الجرد اليومي المقسمة (ProcessDailySalesJob)

- تحميل العلاقات المسبق (Eager Loading): استدعاء Order::with('items') يحل مشكلة الـ N+1 الشهيرة في قواعد البيانات. يتم جلب الطلبات مع عناصرها بطلبين فقط من قاعدة البيانات بدلاً من إطلاق طلب جديد لكل عنصر داخل الحلقة.
- تقسيم الذاكرة المؤقتة (Chunking Strategy): PHP

`->chunk(200, function ($orders) { ... })`

هذا هو المربط الأساسي للأداء العالي. لو كان اليوم يحتوي على 50,000 طلب، فلن يقوم السيرفر بتحميلها دفعة واحدة في الذاكرة (مما يسبب انهيار المخدم)، بل سيقوم بجلب 200 طلب فقط، يعالجها، يفرغ الذاكرة، ثم ينتقل للـ 200 التالية.

- حفظ البيانات الآمن والمقاوم للتكرار (updateOrCreate): يضمن أنه في حال إعادة تشغيل الوظيفة نتيجة خطأ ما، لن تتكرر السجلات في قاعدة البيانات، بل سيتم تحديث السجل الموجود، وهو ما يحقق مفهوم الـ Idempotency.

ثالثاً: وظيفة التجميع الشهري (ProcessMonthlySalesJob)

- التجميع على مستوى قاعدة البيانات (Database Aggregation): بدلاً من قراءة ملايين الطلبات لحساب الشهر، تقرأ هذه الوظيفة سجلات الجرد اليومي الجاهزة فقط (بحد أقصى 31 سجلاً للشهر)، وتستخدم التوابع الرياضية السريعة مثل sum() و count() لتوليد التقرير الشهري بلحظات.

4. أثر الحل على الأداء وإدارة الموارد (Performance Impact)

المعيار	قبل تطبيق الحل (الأسلوب التقليدي المباشر)	بعد تطبيق الحل (Batching & Chunking)
استهلاك الذاكرة (Memory)	يرتفع خطياً مع حجم البيانات وقد يسبب انهيار السيرفر (\$O(N)\$).	ثابت ومستقر دائماً لأن الحد الأقصى في الذاكرة هو 200 سجل فقط (\$O(1)\$).

المعيار	قبل تطبيق الحل (الأسلوب التقليدي المباشر)	بعد تطبيق الحل (Batching & Chunking)
زمن استجابة المستخدم (UX)	ينتظر المستخدم حتى انتهاء العمليات الحسابية .	فوري (Response) في أجزاء من الثانية) لأن العمليات نُقلت بالكامل للخلفية.
الاستفادة من المعالج (CPU)	استهلاك كامل لنواة واحدة بشكل متسلسل (Sequential).	توزيع المهام على عدة أنوية (Parallel) عبر مشغلي الطوابير (Queue Workers).
اعتمادية النظام (Reliability)	أي خطأ في المنتصف يعيد العملية من الصفر ويفسد البيانات.	دعم كامل لإعادة المحاولة (Retries) لكل يوم بشكل مستقل دون التأثير على البقية.

طبعا عم نخط صور من البوست مان لأن عملية الجرد لا تحتاج لمحاكاة على الـ jmeter لعملية الجرد تتم مرة او مرتان او ثلاث في الشهر

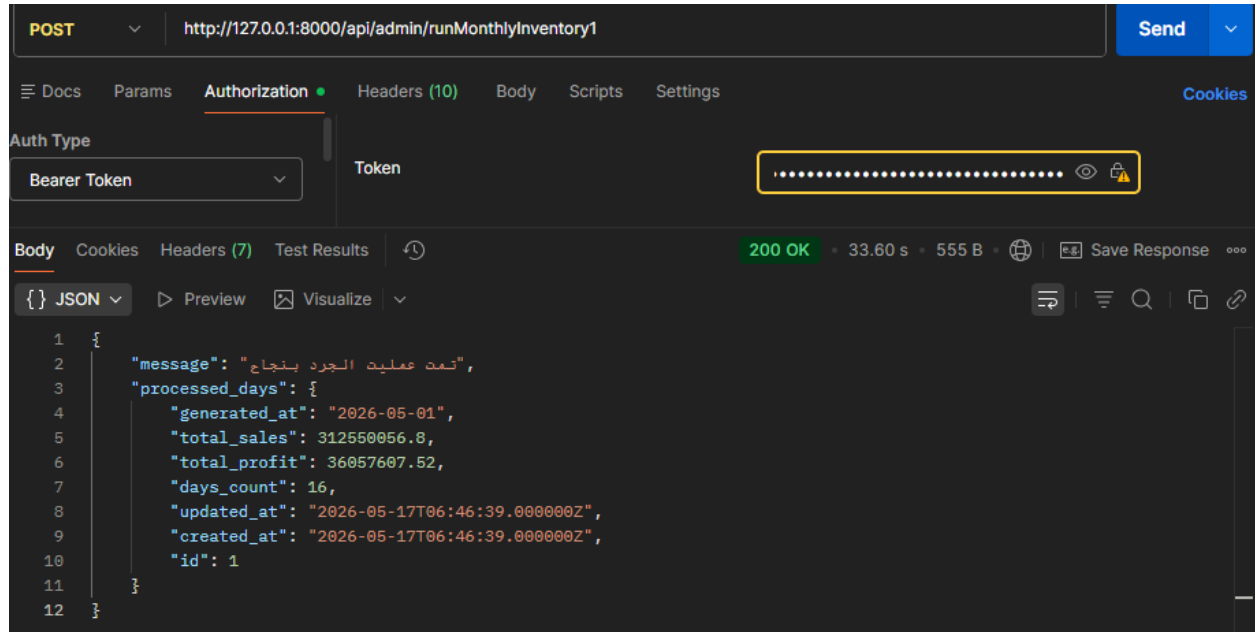
النتيجة قبل تطبيق background job والـ batches والـ chunk:

حيث تم تطبيق الكود ببساطة عبارة عن لوبين متداخلين

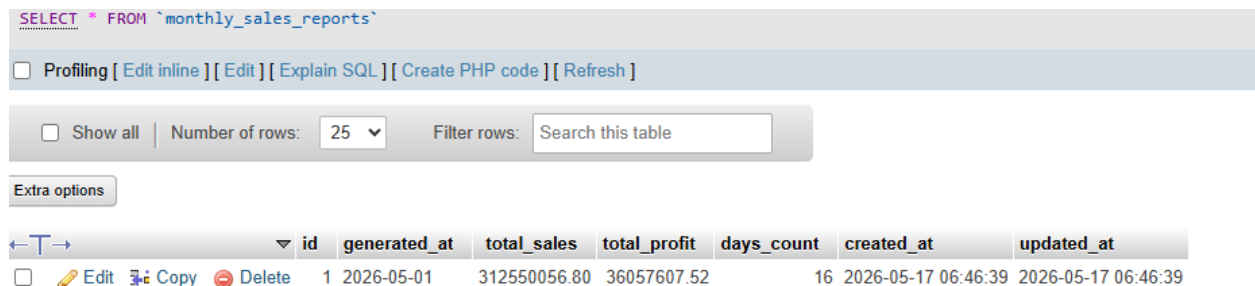
لوب بمر عالأيام ولوب عطربات الأيام وبيقرأ 10000 orders دفعة واحدة من قاعدة البيانات

حيث قمنا بجرده ((160 ألف طلب)) من الشهر الحالي حيث كل يوم يحتوي 10000 طلب و

ملاحظة نحن وقت عملية الجرد كان 17 الشهر
ملاحظة اليوم الاخير 17 يحتوي ع 0 طلب



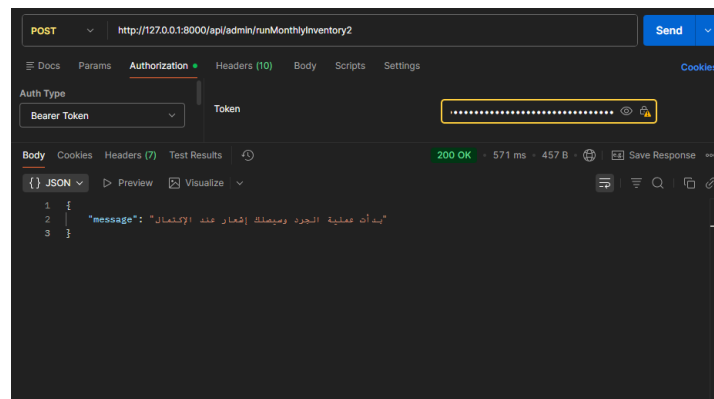
قمنا بوضع صورة للجرد من قاعدة البيانات لأنه سيكون أوضح وإلا فإنه يوجد api لعرض
معلومات الجرد



وبالصورة التالية الجرد اليومي لكل يوم من الشهر

		id	date	total_sales	total_profit	orders_count	created_at	updated_at
☐	Edit Copy Delete	1	2026-05-01	19294492.96	2231592.28	10000	2026-05-17 06:46:08	2026-05-17 06:46:08
☐	Edit Copy Delete	2	2026-05-02	19671658.28	2274029.55	10000	2026-05-17 06:46:10	2026-05-17 06:46:10
☐	Edit Copy Delete	3	2026-05-03	19500284.17	2228944.45	10000	2026-05-17 06:46:12	2026-05-17 06:46:12
☐	Edit Copy Delete	4	2026-05-04	19639668.12	2262445.90	10000	2026-05-17 06:46:14	2026-05-17 06:46:14
☐	Edit Copy Delete	5	2026-05-05	19585530.22	2263803.95	10000	2026-05-17 06:46:16	2026-05-17 06:46:16
☐	Edit Copy Delete	6	2026-05-06	19495852.62	2259978.64	10000	2026-05-17 06:46:18	2026-05-17 06:46:18
☐	Edit Copy Delete	7	2026-05-07	19580919.25	2246034.63	10000	2026-05-17 06:46:20	2026-05-17 06:46:20
☐	Edit Copy Delete	8	2026-05-08	19630180.35	2259074.75	10000	2026-05-17 06:46:22	2026-05-17 06:46:22
☐	Edit Copy Delete	9	2026-05-09	19498980.78	2245769.08	10000	2026-05-17 06:46:24	2026-05-17 06:46:24
☐	Edit Copy Delete	10	2026-05-10	19585909.34	2268214.94	10000	2026-05-17 06:46:27	2026-05-17 06:46:27
☐	Edit Copy Delete	11	2026-05-11	19629774.75	2271245.83	10000	2026-05-17 06:46:28	2026-05-17 06:46:28
☐	Edit Copy Delete	12	2026-05-12	19431993.10	2238802.55	10000	2026-05-17 06:46:31	2026-05-17 06:46:31
☐	Edit Copy Delete	13	2026-05-13	19564148.93	2254586.49	10000	2026-05-17 06:46:32	2026-05-17 06:46:32
☐	Edit Copy Delete	14	2026-05-14	19550930.66	2256308.11	10000	2026-05-17 06:46:35	2026-05-17 06:46:35
☐	Edit Copy Delete	15	2026-05-15	19376966.14	2242209.66	10000	2026-05-17 06:46:36	2026-05-17 06:46:36
☐	Edit Copy Delete	16	2026-05-16	19512767.13	2254566.71	10000	2026-05-17 06:46:39	2026-05-17 06:46:39
☐	Edit Copy Delete	17	2026-05-17	0.00	0.00	0	2026-05-17 06:46:39	2026-05-17 06:46:39

وبعد استعمال ال background job وال batches وال chunk:



```
SELECT * FROM `monthly_sales_reports`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

		id	generated_at	total_sales	total_profit	days_count	created_at	updated_at
☐	Edit Copy Delete	1	2026-05-01	312550056.80	36057607.52	16	2026-05-17 08:23:25	2026-05-17 08:23:25

Server: 127.0.0.1 » Database: ecommerce » Table: daily_sales_reports

Browse

Structure

SQL

Search

Insert

Export

Import

Privileges

Operations

☐ Profiling

[\[Edit inline \]](#)

[\[Edit \]](#)

[\[Explain SQL \]](#)

[\[Create PHP code \]](#)

[\[Refresh \]](#)

☐ Show all

Number of rows:

25

Filter rows:

Search this table

Sort by key:

None

Extra options

←

→

▼

id

date

total_sales

total_profit

orders_count

created_at

updated_at

☐

1

2026-05-02

19671658.28

2274029.55

10000

2026-05-17 08:23:04

2026-05-17 08:23:04

☐

2

2026-05-01

19294492.96

2231592.28

10000

2026-05-17 08:23:04

2026-05-17 08:23:04

☐

3

2026-05-03

19500284.17

2228944.45

10000

2026-05-17 08:23:06

2026-05-17 08:23:06

☐

4

2026-05-04

19639668.12

2262445.90

10000

2026-05-17 08:23:07

2026-05-17 08:23:07

☐

5

2026-05-05

19585530.22

2263803.95

10000

2026-05-17 08:23:08

2026-05-17 08:23:08

☐

6

2026-05-06

19495852.62

2259978.64

10000

2026-05-17 08:23:09

2026-05-17 08:23:09

☐

7

2026-05-07

19580919.25

2246034.63

10000

2026-05-17 08:23:11

2026-05-17 08:23:11

☐

8

2026-05-08

19630180.35

2259074.75

10000

2026-05-17 08:23:12

2026-05-17 08:23:12

☐

10

2026-05-09

19498980.78

2245769.08

10000

2026-05-17 08:23:15

2026-05-17 08:23:15

☐

11

2026-05-10

19585909.34

2268214.94

10000

2026-05-17 08:23:18

2026-05-17 08:23:18

☐

12

2026-05-11

19629774.75

2271245.83

10000

2026-05-17 08:23:19

2026-05-17 08:23:19

☐

13

2026-05-12

19431993.10

2238802.55

10000

2026-05-17 08:23:21

2026-05-17 08:23:21

☐

14

2026-05-13

19564148.93

2254586.49

10000

2026-05-17 08:23:22

2026-05-17 08:23:22

☐

15

2026-05-14

19550930.66

2256308.11

10000

2026-05-17 08:23:23

2026-05-17 08:23:23

16

2026-05-15

19376966.14

2242209.66

10000

2026-05-17 08:23:24

2026-05-17 08:23:24

المعيار	قبل تطبيق أفكار الطالب	بعد تطبيق أفكار الطالب
طريقة التنفيذ	تنفيذ مباشر داخل request واحد	تقسيم العمل إلى jobs + batch queue
التوازي (Parallelism)	لا يوجد	نعم (حتى 9 workers)
استخدام CPU	25% ← 70% (غير مستغل بالكامل)	60% ← 100% (استغلال كامل للـ cores)
استهلاك RAM (peak)	من 3 ل 5 GB دفعة واحدة	من 1.5 ل 4.5 GB موزعة على workers
شكل استهلاك RAM	Spike واحد كبير	توزيع على عدة عمليات
Response Time	تقريبا من 20 ل 120 sec	من 200 ل 700 ميلي ثانسه
زمن معالجة k160 طلب	طويل (واحد ثقيل)	أسرع (Parallel execution)

المعيار	قبل تطبيق أفكار الطلب	بعد تطبيق أفكار الطلب
I/O Pressure على DB	عالي جدًا	متوسط (لكن متوازي)
مرونة النظام	منخفضة	عالية جدًا
إدارة الأخطاء	صعبة (fail = restart كامل)	سهلة (job-level retry)
Retry عند الفشل	غير عملي	مدمج في queue system
تأثير على السيرفر	ضغط لحظي عالي	ضغط مستمر لكن موزع
UX (تجربة المستخدم)	سيئة (انتظار طويل)	ممتازة (async processing)

الطلب الخامس

تم الاعتماد على التقنيات التالية:

- NGINX كموزع أحمال (Load Balancer)
- إطار العمل Laravel لبناء الخوادم الخلفية
- تشغيل عدة خوادم Laravel على منافذ مختلفة 8001 ، 8002 ، 8003
- استخدام HTTP Reverse Proxy لتمرير الطلبات بين الخادم الوسيط والخوادم الخلفية

استراتيجية توزيع الأحمال المستخدمة:

تم استخدام استراتيجية هجينة (Hybrid Strategy) تجمع بين:

Least Connections أقل عدد اتصالات:

يتم توجيه الطلب إلى الخادم الذي يمتلك أقل عدد من الاتصالات النشطة، مما يساعد على تقليل الضغط على الخوادم المكتظة وتحسين سرعة الاستجابة.

Weighted Load Balancing التوزيع الموزون:

يتم إعطاء كل خادم وزناً (Weight) يعبر عن قدرته على معالجة الطلبات. الخوادم الأقوى تحصل على عدد أكبر من الطلبات مقارنة بالخوادم الأضعف.

آلية العمل

تتم عملية توزيع الطلبات وفق الخطوات التالية:

1. استقبال الطلب من المستخدم عبر NGINX
2. تحليل حالة جميع الخوادم الخلفية
3. حساب أفضل خادم باستخدام المعادلة :

- $\text{Score} = \text{Current Connections} / \text{Weight}$

4. اختيار الخادم الذي يحقق أقل قيمة
5. تمرير الطلب إلى الخادم المختار
6. إعادة الاستجابة إلى المستخدم

الهدف من الاستراتيجية

تم اعتماد هذه الاستراتيجية لتحقيق:

- تحسين توزيع الحمل بين الخوادم
- منع ضغط زائد على خادم واحد

- تحسين زمن الاستجابة (Response Time)
- زيادة قابلية التوسع (Scalability)
- تحسين استقرار النظام في حال زيادة عدد الطلبات

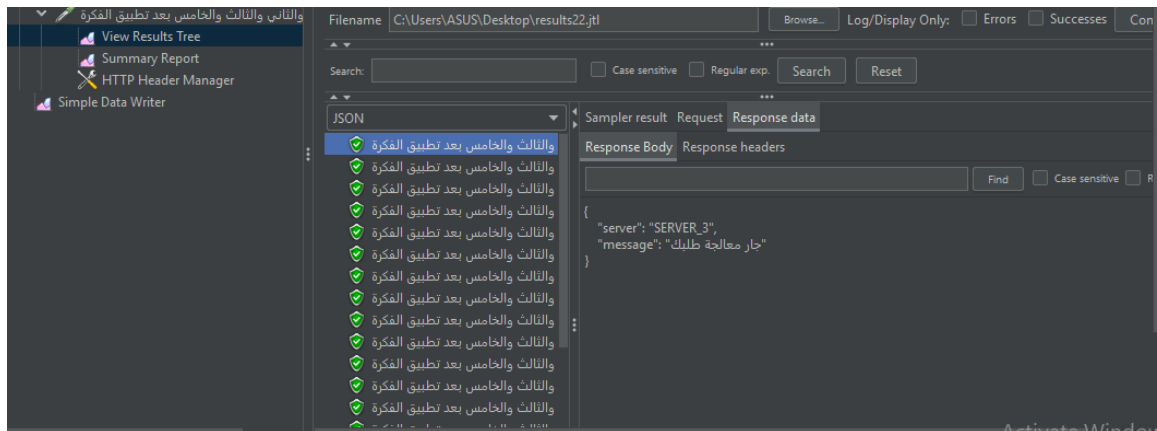
بعد تطبيق النظام ومجربته محليًا:

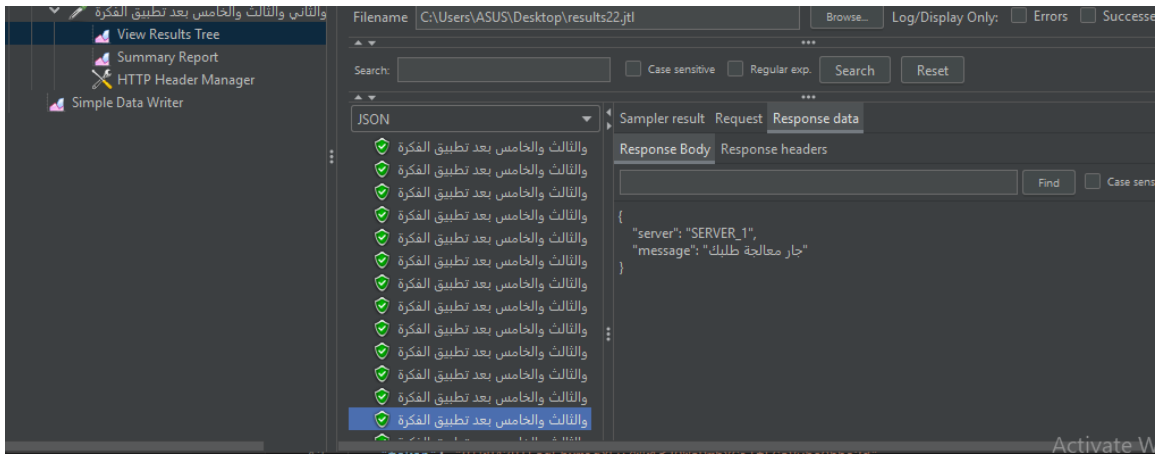
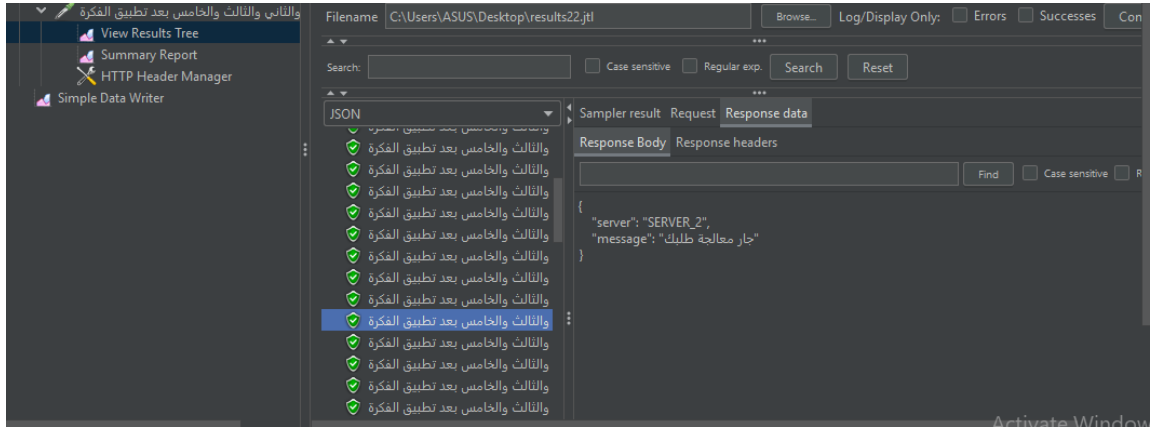
- تم توزيع الطلبات بشكل متوازن بين الخوادم
- الخوادم ذات الوزن الأعلى استقبلت عدد طلبات أكبر
- تم تقليل الضغط على خادم واحد فقط
- النظام أظهر قدرة جيدة على التعامل مع عدد من الطلبات المتزامنة حتى ~100 request بشكل مستقر في البيئة المحلية

الخلاصة:

تم تنفيذ نظام Load Balancing محاكي باستخدام NGINX و Laravel يعتمد على استراتيجية هجينة بين Least Connections و Weighted Distribution. أظهرت النتائج أن هذه الاستراتيجية فعالة في تحسين الأداء وتوزيع الأحمال بشكل عادل وذكي، مما يجعلها مناسبة للأنظمة ذات الطلبات العالية مثل تطبيقات التجارة الإلكترونية.

وهذه بعض الصور المختارة من محاكاة 100 ريكوست وتبين الصور توزيع الحمل على الثلاث سيرفرات





الموارد لدي: 8ram و 8cores

وسيتم تطبيق محاكاة عال jmeter على دالة createOrder التي تطبق queue job

الموارد قبل تطبيق الاستراتيجية

العنصر	القيمة
استهلاك المعالج CPU	68% — 78%
استهلاك الذاكرة RAM	3.8 GB — 4.6 GB

الموارد بعد تطبيق الاستراتيجية:

العنصر	القيمة
استهلاك المعالج CPU	45-60%
استهلاك الذاكرة RAM	3.2-4.0GB

بعد تطبيق استراتيجية توزيع الأحمال، لوحظ ما يلي:

- انخفاض استهلاك المعالج بنسبة ملحوظة (حوالي 15% - 20%)
- انخفاض استهلاك الذاكرة RAM بشكل عام
- تحسن توزيع الضغط بدلاً من تركزه في نقطة واحدة
- زيادة قدرة النظام على التعامل مع الطلبات المتزامنة

أثبتت النتائج أن تطبيق استراتيجية توزيع الأحمال باستخدام NGINX و عدة خوادم Laravel أدى إلى تحسين أداء النظام بشكل واضح، من خلال تقليل الضغط على الموارد وتحسين استقرار الأداء أثناء تنفيذ عدد كبير من الطلبات المتزامنة.